



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

Facultad de Ciencias

Compiladores

Fases de Compilación y expresiones regulares

Tarea 1

Presenta:

Ramírez Juárez María Fernanda - 321204747

Rojo Peña Manuel Ianluck - 118005762

Profesor:

Pablo Martínez Yessica Janeth

Ayudante:

Rojas Fuentes Andrea

Grupo: 7010, 2027-1

Fecha de entrega:

4 de septiembre, 2026

1. Con lo visto en clase y **con tus propias palabras**, completa la siguiente tabla. Puedes ayudarte también de alguna fuente académica (libros, artículos); de ser así, agregar referencias.

Fase	Otro nombre	Función	Resultado o salida	Tabla de símbolos	Manejo de errores	Ejemplo
Análisis léxico	Escáner Scanning	En esta parte se lee la cadena de caracteres del programa y los agrupa en lexemas	Regresa un flujo de fichas léxicas o <i>tokens</i> con la forma <nombre_token, valor_atributo>	Detecta e introduce por primera vez los nombres de identificadores (variables, funciones) en la tabla	En esta parte se reportan los caracteres no válidos en el alfabeto del lenguaje, cadenas, comentarios sin cerrar	total = suma + 10 lexema Token total <ID,1> = <OP,=> suma <ID,2> + <OP,+> 10 <NUM,10>
Análisis sintáctico	Parsing	Revisa que la secuencia de tokens entregada por el analizador léxico cumpla con las reglas estructurales del lenguaje	Regresa un árbol de sintaxis abstracta (AST) ó árbol de análisis sintáctico	Consulta y añade información sobre el ámbito (<i>scope</i>) y las estructuras de control	Se detectan los token que estan "fuera de lugar" es decir revisa si fatan paréntesis,coma s o puntos y comas omitidos	Con la gramatica: S -> L = E L -> id[E] E -> E + E id num Obtener el AST de : a[i]=4+2 <pre> graph TD S --> L S --> E1[E] L --> id1[id] E1 --> E2[E] E1 --> eq[=] E2 --> id2[id] E2 --> num[num] num --> 4 num --> 2 </pre>

Análisis semántico	Analizador contextual	En esta parte se utiliza la estructura obtenida en el paso anterior a partir de ella se revisa la consistencia del significado del programa revisando la compatibilidad de tipos y la información de semántica	Esta parte regresa Árbol sintáctico anotado con tipos de datos e información semántica	Actualiza la tabla asociando tipos de datos, tamaños y firmas de funciones a cada identificador	Se reportan tipos incompatibles , variables no declaradas o llamadas a funciones con número incorrecto de argumentos	<pre> graph TD S --> L S --> E1[E] L --> id1[id] E1 --> E2[E] E1 --> eq[=] E2 --> id2[id] E2 --> num[num] num --> 4 num --> 2 </pre>
Generador de código intermedio	Representación intermedia	En esta parte se crea un lenguaje puente que une la fase donde el compilador entiende el programa con la fase de crear el código final para la computadora	La salida de esta fase es una versión simplificada de las instrucciones convirtiéndolas en pasos básicos y estándar	Asigna y consulta ubicaciones temporales y desplazamientos de memoria	Se detectan errores al convertir el código o avisa si el compilador se queda sin casillas (espacio) temporales para guardar datos mientras procesa el programa	Expresión inicial: x = a + b * c Código intermedio: t1 = b * c x = a + t1

Optimizador de código independiente de la máquina	-	En esta parte se modifica la representación intermedia para que el programa ejecute más rápido o consuma menos memoria, sin considerar el	Esta parte regresa una representación intermedia optimizada y más eficiente para que el programa ejecute más	Elimina variables temporales en desuso y limpia entradas de código muerto	Aquí se advierte sobre código inalcanzable (dead code) o variables inicializadas sin uso	Si tenemos la expresión 5 * 2 el optimizador la reemplaza directamente por 10 antes de ejecutar
---	---	---	--	---	--	---

		procesador final y sin cambiar el comportamiento del programa	rápido o consume menos memoria, sin considerar el procesador final			
Generador de código	Generador de código máquina.	Significa traducir la versión optimizada del código intermedio del programa a los comandos exactos que entiende el chip físico donde va a funcionar	Se regresa el código en lenguaje ensamblador o código objeto relocatable	Determina la dirección física o el desplazamiento exacto en memoria de cada variable y función	Revisa la falta de registros físicos suficientes en la CPU o sobrepaso de límites de memoria de direccionamiento	LDF R2, id2 ADDF R2, R2, #10.0 STF id1, R2
Optimizador de código independiente de la máquina	Optimizador de bajo nivel	Reorganiza y sustituye las instrucciones en ensamblador para sacar el máximo provecho de las características físicas de la CPU	Como resultado final tenemos el código máquina final binario o ensamblador altamente optimizado	Genera tablas de depuración si el compilador las requiere	Revisa violaciones de alineación de memoria del procesador o conflictos de canalización	Reemplazar una multiplicación por 2 por un desplazamiento de bits hacia la izquierda (SHL R1, 1), que requiere menos ciclos de reloj.

Para mejor vista de la tabla visitar:

https://docs.google.com/document/d/1NykWZG_F60oQRo1WW8hRM8S2H200KHGM0qQ1cKzUPCo/edit?usp=sharing

2. Considere la siguiente gramática:

$$\begin{aligned} S &\rightarrow L = E; \\ L &\rightarrow \text{id}[E] \mid L[E] \\ E &\rightarrow E + E \mid E * E \mid (E) \mid \text{id} \mid \text{num} \end{aligned}$$

y la siguiente instrucción:

```
int a[3], b, i;
a[i] = b + 4 * 2;
```

Realiza el proceso de compilación siguiendo todas las fases así como se vio en clase (describe detalladamente cada una de ellas).

Tokens Generados:

```
<id, "a">
<corA, "[">
<id, "i">
<corC, "]">
<asig, "=">
<id, "b">
<sum, "+">
<num, "4">
<mul, "*">
<num, "2">
<finInstr, ";">
```

Tabla de Símbolos Generada:

Suponemos que `int`: 4 bytes:

`a[3]` => $3 \times 4 = 12$ `b, j` => 4 bytes cada uno.

Posición	ID	Tipo	Total Bytes	Dirección
1	a	int	12	0
1	b	int	4	12
1	c	int	4	16

Árbol de Sintaxis Abstracta Generado:

fortree = *mathcontent*, *edge* = −, *ssep* = 10pt, *lsep* = 12pt, *align* = center, $[a[i] = b + 4 * 2; [L = E; [a[i][i$

Código Intermedio Generado:

```
a[i] = b + 4 * 2
t1 = 4 * 2      //Aislamos la multiplicacion
t2 = b + t1     //Aislamos la suma
t3 = i * 4      //Calculamos offset en memoria
a[t2] = t1      //Asignamos la dir base de a + offset
```

Código Intermedio Optimizado:

```
a[i] = b + 4 * 2
t1 = b + 8      //El compilador al ver los dos numeros
                // fijos, los evalua de antemano
t2 = i << 2      //Multiplicar por potencias de dos
                // es identico a recorrer bit a la izq
a[t2] = t1
```

3. Dada la siguiente expresión regular, completa la tabla con $\checkmark$ o $\times$ y muestra el procedimiento de tu respuesta.

$$(a \mid b)^* a (a \mid b) (a \mid b).$$

CADENA	¿ACEPTADA?	PROCEDIMIENTO O JUSTIFICACIÓN
abaaabb	$\times$	No hace falta comprobar toda la cadena, basta con notar que los primeros dos caracteres son a y b , no son posibles con esa expresión regular, ya que $a \mid b^*$, solo genera cadenas de uno o más símbolos <i>a</i> , o una o más símbolos <i>b</i> , o la cadena vacía y solo esos. Lo que hace que no sea posible que la regex genere la cadena <i>ab</i> como prefijo. Por otro lado, si ignoramos esta primera sección, la longitud de la cadena que estamos evaluando excede los caracteres que pueden generarse con el resto de expresión, puesto que $a(a \mid b)(a \mid b)$ genera subcadenas de tres caracteres, no de los siete que tiene la cadena dada.
babb	$\checkmark$	Esta cadena sí es aceptada, ya que la primera <i>b</i> proviene de la <i>b</i> repetida una sola vez de $a \mid b^*$ y el resto de la cadena <i>abb</i> , se genera con $a(a \mid b)(a \mid b)$ eligiendo el caracter <i>b</i> en ambas alternancias.
baaaab	$\times$	Como se puede ver, la cadena cuenta con 4 letras <i>a</i> , después de un caracter <i>b</i> , y como anteriormente mencionamos, el final de la expresión solo puede generar cadenas de a lo más tres caracteres, por lo que generar cuatro <i>a</i> 's después de una <i>b</i> , no es posible para esta regex.
aaabaab	$\times$	No es aceptada ya que, si en $(a \mid b)^*$, lo usamos para generar los primeros dos símbolos <i>a</i> y el tercero con el <i>a</i> concatenado en medio de la regex, ya no es posible generar el resto de la cadena <i>baab</i> , ya que con los componentes $(a \mid b)(a \mid b)$, no podemos generar esos cuatro símbolos, solo dos.
ϵ	$\times$	No es posible que la cadena vacía sea aceptada por esta regex, ya que el símbolo <i>a</i> después de $(a \mid b)^*$ nos restringe a que mínimo tenga un caracter. Y de nuevo, como anteriormente mencionamos $(a \mid b)(a \mid b)$ obliga a que se elija alguno de los dos símbolo en ambas opciones, por lo que la regex mínimo genera cadenas de tres caracteres, nunca una vacía.

4. Escribe una expresión regular para cada inciso, con el siguiente alfabeto $\Sigma = \{a, b, c\}$ que:

a. Contengan un número **impar** de letras a (considerando desde 0):

$$((ab \mid ca)(aa \mid bb \mid cc))^*$$

b. Tengan **longitud impar**:

$$(b \mid a \mid c)(cc \mid aa \mid bb)^*$$

c. No contengan la subcadena bb :

$$(ba \mid bc)^*(ab \mid cb)^+$$

Da una expresión regular para todas las cadenas de letras en minúsculas que contengan las cinco vocales en orden.

$$(a \mid b \mid \dots \mid y \mid z)^* a (a \mid b \mid \dots \mid y \mid z)^* e (a \mid b \mid \dots \mid y \mid z)^* i (a \mid b \mid \dots \mid y \mid z)^* \\ o (a \mid b \mid \dots \mid y \mid z)^* u (a \mid b \mid \dots \mid y \mid z)^*$$

5. Utiliza el algoritmo de elementos puntuados para resolver lo siguiente (explica detalladamente el proceso así como se vio en clase):

a. $r \rightarrow b(b \mid aa^*b)c$

Cerradura($\{ r \rightarrow \bullet b(b \mid aa^*b)c \}$) } No hace nada pues b , es símbolo básico.

Estado:

$$q_0 = \{ r \rightarrow \bullet b(b \mid aa^*b)c \}$$

Transiciones:

$$\begin{aligned} \text{goto}(q_0, b) &= \{ r \rightarrow b \bullet (b \mid aa^*b)c \\ &\quad r \rightarrow b(\bullet b \mid aa^*b)c \\ &\quad r \rightarrow b(b \mid \bullet aa^*b)c \} = q_1 \\ \text{goto}(q_0, a) &= \emptyset \\ \text{goto}(q_0, c) &= \emptyset \end{aligned}$$

Estado:

$$\begin{aligned} q_1 &= \{ r \rightarrow b \bullet (b \mid aa^*b)c \\ &\quad r \rightarrow b(\bullet b \mid aa^*b)c \\ &\quad r \rightarrow b(b \mid \bullet aa^*b)c \} \end{aligned}$$

Transiciones:

$$\begin{aligned} \text{goto}(q_1, b) &= \{ r \rightarrow b(\bullet b \mid aa^*b)c \\ &\quad r \rightarrow b(b \bullet \mid aa^*b)c \\ &\quad r \rightarrow b(b \mid aa^*b) \bullet c \} = q_2 \\ \text{goto}(q_1, a) &= \{ r \rightarrow b(b \mid \bullet aa^*b)c \\ &\quad r \rightarrow b(b \mid a \bullet a^*b)c \\ &\quad r \rightarrow b(b \mid aa^* \bullet b)c \} = q_3 \\ \text{goto}(q_1, c) &= \emptyset \end{aligned}$$

Estado:

$$\begin{aligned} q_3 &= \{ r \rightarrow b(b \mid a \bullet a^*b)c \\ &\quad r \rightarrow b(b \mid aa^* \bullet b)c \} \end{aligned}$$

Transiciones:

$$\begin{aligned} \text{goto}(q_3, a) &= \{ r \rightarrow b(b \mid a \bullet a^*b)c \\ &\quad r \rightarrow b(b \mid aa^* \bullet b)c \} = q_3 \\ \text{goto}(q_3, b) &= \{ r \rightarrow b(b \mid aa^*b \bullet)c \\ &\quad r \rightarrow b(b \mid aa^*b) \bullet c \} = q_2 \\ \text{goto}(q_3, c) &= \emptyset \end{aligned}$$

Estado:

$$\begin{aligned} q_2 &= \{ r \rightarrow b(b \bullet \mid aa^*b)c \\ &\quad r \rightarrow b(b \mid aa^*b \bullet)c \\ &\quad r \rightarrow b(b \mid aa^*b) \bullet c \} \end{aligned}$$

Transiciones:

$$\text{goto}(q_2, c) = \{ r \rightarrow b(b \mid aa^*b) \bullet c \\ r \rightarrow b(b \mid aa^*b)c \bullet \} = q_4$$

$$\text{goto}(q_2, b) = \emptyset$$

$$\text{goto}(q_2, a) = \emptyset$$

Estado:

$$q_4 = \{ r \rightarrow b(b \mid aa^*b)c \bullet \}$$

Estado de aceptación.

Transiciones:

$$\text{goto}(q_4, a) = \emptyset$$

$$\text{goto}(q_4, b) = \emptyset$$

$$\text{goto}(q_4, c) = \emptyset$$

Tabla de transiciones:

	<i>b</i>	<i>a</i>	<i>c</i>
$\rightarrow q_0$	q_1	$\emptyset$	$\emptyset$
q_1	q_2	q_3	$\emptyset$
q_2	$\emptyset$	$\emptyset$	q_4
q_3	q_2	q_3	$\emptyset$
*q_4	$\emptyset$	$\emptyset$	$\emptyset$

b. $r \rightarrow c(a \mid b)^*$

Cerradura($\{ r \rightarrow \bullet c(a \mid b)^* \}$) } No hace nada pues c , es símbolo básico.

Estado:

$$q_0 = \{ r \rightarrow \bullet c(a \mid b)^* \}$$

Transiciones:

$$\begin{aligned} \text{goto}(q_0, c) &= \{ r \rightarrow c \bullet (a \mid b)^* \\ &\quad r \rightarrow c(\bullet a \mid b)^* \\ &\quad r \rightarrow c(a \mid \bullet b)^* \\ &\quad r \rightarrow c(a \mid b)^* \bullet \} = q_1 \end{aligned}$$

$$\text{goto}(q_0, a) = \emptyset$$

$$\text{goto}(q_0, b) = \emptyset$$

Estado:

$$\begin{aligned} q_1 &= \{ r \rightarrow c(\bullet a \mid b)^* \\ &\quad r \rightarrow c(a \mid \bullet b)^* \\ &\quad r \rightarrow c(a \mid b)^* \bullet \} \end{aligned}$$

Estado de aceptación.

Transiciones:

$$\text{goto}(q_1, c) = \emptyset$$

$$\begin{aligned} \text{goto}(q_1, a) &= \{ r \rightarrow c(\bullet a \mid b)^* \\ &\quad r \rightarrow c(a \bullet \mid b)^* \\ &\quad r \rightarrow c(a \mid b)^* \bullet \} = q_1 \end{aligned}$$

$$\begin{aligned} \text{goto}(q_1, b) &= \{ r \rightarrow c(a \mid \bullet b)^* \\ &\quad r \rightarrow c(a \mid b \bullet)^* \\ &\quad r \rightarrow c(a \mid b)^* \bullet \} = q_1 \end{aligned}$$

Tabla de transiciones:

	c	a	b
$\rightarrow q_0$	q_1	$\emptyset$	$\emptyset$
$^* q_1$	$\emptyset$	q_1	q_1